Lab 2: Modelling

Ashley Walsh (440295983)

2026-08-04

Setup

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

Part A: Tracer decay and step size (numerical)

This code is produces a tracer-decay model, and visually overlays the numerical model curve and the exact curve.

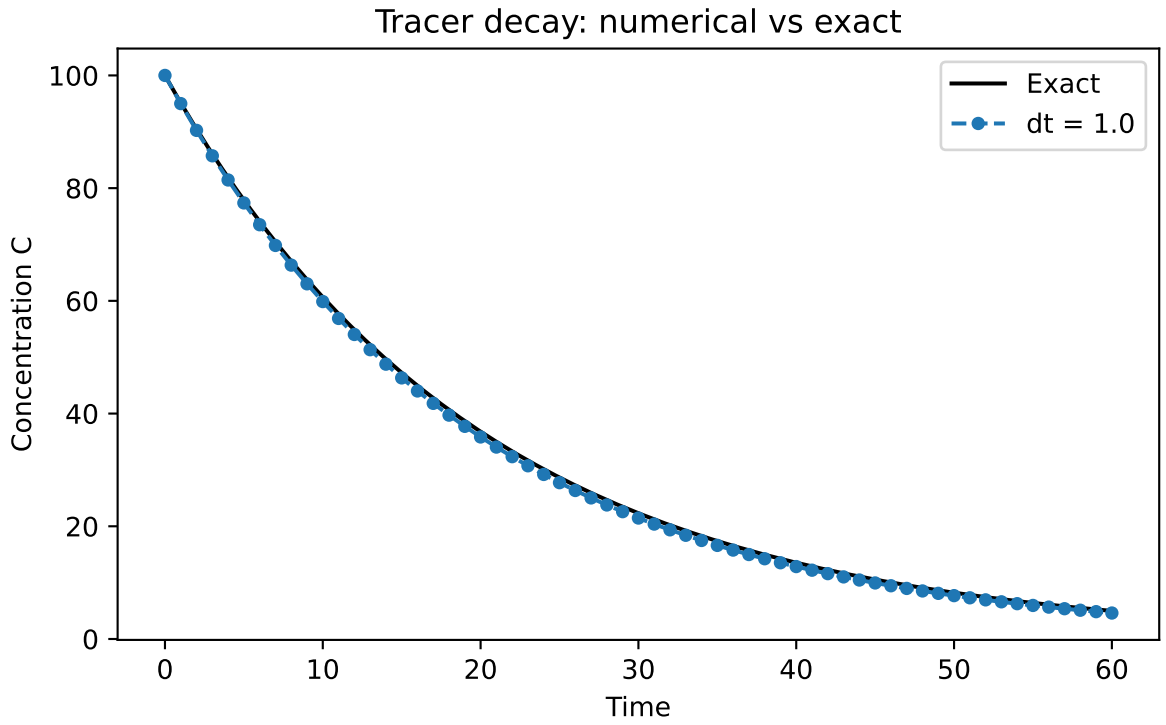
```
# Create function for numerical model
def decay_numerical(C0, k, dt, t_end):
    """Return arrays of time and concentration for forward-Euler tracer decay."""
    n = int(t_end / dt)
    C = C0
    ts = [0.0]
    Cs = [C0]
    for i in range(n):
        C = C + (-k * C) * dt          # forward difference
        ts.append((i + 1) * dt)
        Cs.append(C)
    return np.array(ts), np.array(Cs)

# The exact answer, for comparison
C0, k, t_end = 100.0, 0.05, 60
t_fine = np.linspace(0, t_end, 200)
exact = C0 * np.exp(-k * t_fine)
```

```python
# Create plot
fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(t_fine, exact, "k-", label="Exact")

for dt in [1.0]:
    ts, Cs = decay_numerical(C0, k, dt, t_end)
    ax.plot(ts, Cs, "o--", markersize=4, label=f"dt = {dt}")

ax.set_xlabel("Time")
ax.set_ylabel("Concentration C")
ax.set_title("Tracer decay: numerical vs exact")
ax.legend()
plt.show()
```



Answer to task A1:

In this model `k` is the decay constant which changes the rate at which the tracer concentration decreases over time, effectively determining the curvature of the slope, and how quickly it becomes undetectable in the lake.

Answer to task A2:

The `dt` begins to disagree with the exact curve when `dt = 2.0`, although only slightly, and becomes very visually disagreeable at `dt = 5.0`.

Answer to task A3:

The largest `dt` that still gives a non-negative curve is `dt = 1.0`, where the concentration hits zero at time "1" but does not decrease into negatives, as seen on the figure below.

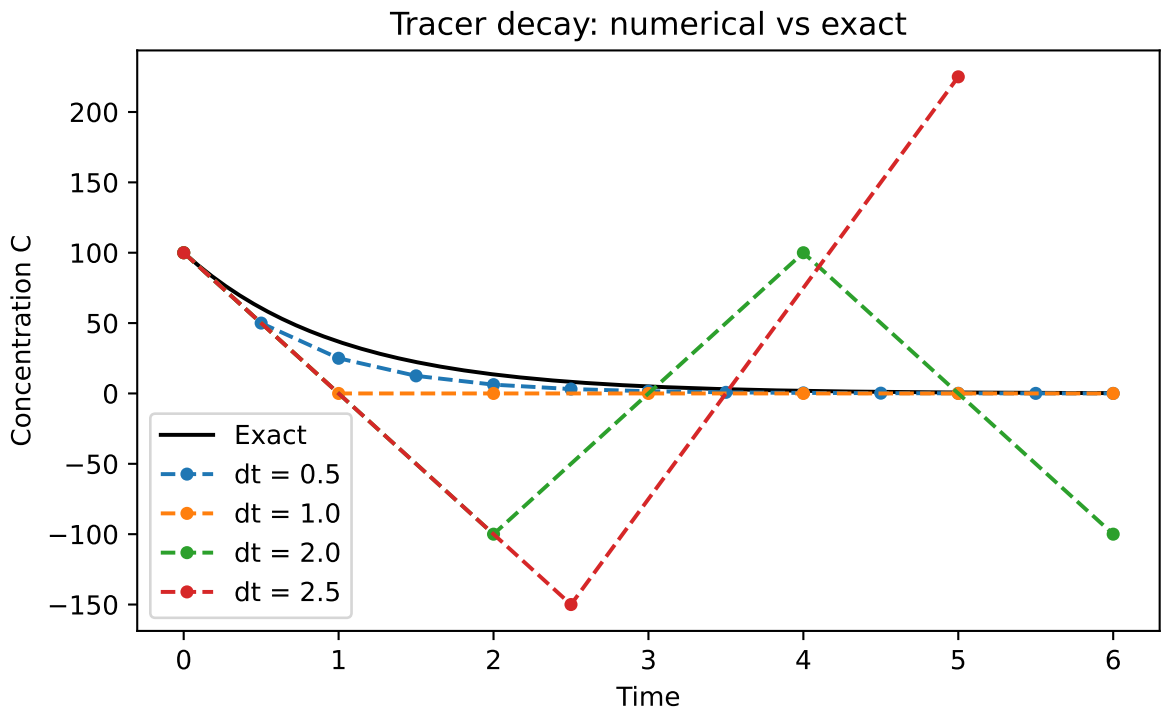
```
# Visually compare `dt` numerical model values

# The exact answer, for comparison
C0, k, t_end = 100.0, 1.0, 6.0
t_fine = np.linspace(0, t_end, 200)
exact = C0 * np.exp(-k * t_fine)

# Create plot
fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(t_fine, exact, "k-", label="Exact")

for dt in [0.5, 1.0, 2.0, 2.5]:
    ts, Cs = decay_numerical(C0, k, dt, t_end)
    ax.plot(ts, Cs, "o--", markersize=4, label=f"dt = {dt}")

ax.set_xlabel("Time")
ax.set_ylabel("Concentration C")
ax.set_title("Tracer decay: numerical vs exact")
ax.legend()
plt.show()
```



Answer to task A4:

At each step, the model uses $C_{old} - kC_{old}dt$ to calculate the decrease in concentration by multiplying C_{old} (100), the decay constant k (1), and the dt value. Any value where $k * dt > 1$ will result in a number larger than the original starting concentration, shooting the value into the negatives (e.g. $100 - 1 * 100 * 1 = 0$ versus $100 - 1 * 100 * 2 = -100$), which we know are physically meaningless for this model in reality.

Part B: Reservoir box model (process-based)

This code produced a process-based 'reservoir box model' that models the concentration of pollution inside a lake fed by a river.

```
# ---- Parameters ----
V = 1.0e6         # lake volume (m^3)
Q = 5.0e3         # river inflow (m^3 per day)
C_in = 10.0       # incoming concentration (mg/L)
k = 0.05          # in-lake decay rate (per day)

# ---- Time settings ----
dt = 1.0          # time step (days)
```

```
days = 200

# ---- Initial condition ----
C = 0.0          # clean lake

# ---- Storage ----
history = [C]

# ---- Time loop ----
for _ in range(days):
    dCdt = (Q * C_in - Q * C - k * V * C) / V   # the rate of change
    C = C + dCdt * dt                           # forward difference
    history.append(C)

# ---- A quick look at the numbers ----
print(f"Started at {history[0]:.2f} mg/L")
print(f"After 10 days: {history[10]:.2f} mg/L")
print(f"After 50 days: {history[50]:.2f} mg/L")
print(f"After {days} days: {history[-1]:.2f} mg/L")
```

```
Started at 0.00 mg/L
After 10 days: 0.39 mg/L
After 50 days: 0.86 mg/L
After 200 days: 0.91 mg/L
```

Answer to task B1

The analytical steady state $((Q * C_{in})/(Q + k * V))$ calculated below gives the same value output as the numerical model value - both being 0.91 mg/L.

```
# compare analytical & numerical model values

C_star = (Q * C_in) / (Q + k * V)   # analytical steady state equation
print(f"Analytical steady state: {C_star:.2f} mg/L")
print(f"Numerical final value:   {history[-1]:.2f} mg/L")
print(f"Difference:              {abs(C_star - history[-1]):.4f} mg/L")
```

```
Analytical steady state: 0.91 mg/L
Numerical final value:   0.91 mg/L
Difference:              0.0000 mg/L
```

Answer to task B2

The concentration of pollution decreases exponentially over time, as opposed to increasing, and levels off at the same steady state as the "clean lake" model, where inflow balances the outflow and in-lake pollution decay.

Answer to task B3

Doubling Q results in a rise in steady state because Q is multipled by '$C_{in} = 10$' in the numerator position and only added to 'kv' in the denominator position, making the numerator grow faster in the resulting fraction.

Answer to task B4

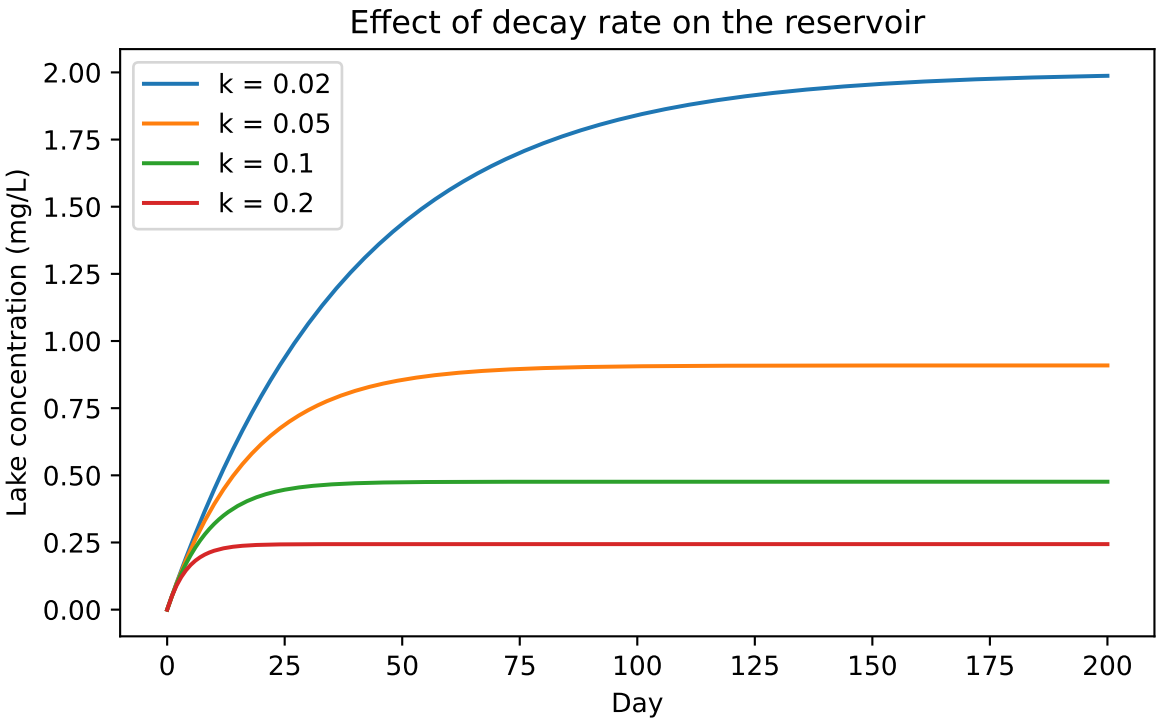
A higher `k` decreases the steady-state value of the lake concentration and increase the speed at which it reaches that point, as seen on the figure below.

```
# Sweep decay rate over four values and compare visually

# Create plot
fig, ax = plt.subplots(figsize=(7, 4))

for k_test in [0.02, 0.05, 0.1, 0.2]:
    C = 0.0
    history = [C]
    for _ in range(days):
        dCdt = (Q * C_in - Q * C - k_test * V * C) / V
        C = C + dCdt * dt
        history.append(C)
    ax.plot(history, label=f"k = {k_test}")

ax.set_xlabel("Day")
ax.set_ylabel("Lake concentration (mg/L)")
ax.set_title("Effect of decay rate on the reservoir")
ax.legend()
plt.show()
```



Reflection

I think the overall process of coding is not too unfamiliar to me but I definitely got stuck on the "equations and formula" style questions, as I find logical mathematics hard to get my head around. Sometimes it helped to visualise this on a plot during the process even if the task itself didn't require one. If I had more time, one change I would make to the code would be to consider better #comments so the clarity is increased for reproducibility.

Checklist before you push

- ☐ Every task has a code cell **and** a short prose answer
- ☐ Every figure has axis labels and a title
- ☐ Your name and student ID are in the YAML header
- ☐ The file renders cleanly (no red error boxes in the HTML output)
- ☐ Filename is `lab-XX-topic.qmd`, not `Untitled.qmd`